

Fraud Detection Analytics using AWS Athena and AWS Glue

Business Scenario

A **FinTech company** processes millions of credit card transactions daily. The raw transaction data is stored in **Amazon S3 as CSV files**.

The fraud analytics team wants to build a **serverless analytics system** to:

- Analyze fraud patterns
- Identify suspicious transactions
- Generate fraud reports automatically

Your task is to use **AWS Glue and AWS Athena** to analyze the dataset and build an **automated analytics pipeline**.

Dataset

Link: <https://www.kaggle.com/datasets/whenamancodes/fraud-detection>

CSV files stored in S3:

s3://fraud-detection-datalake/transactions/

Dataset columns:

Column	Description
Time	Time elapsed between transactions
V1–V28	PCA transformed anonymized features
Amount	Transaction amount
Class	Fraud label (0 = normal, 1 = fraud)

Example

Time,V1,V2,V3,V4,V5,V6,V7,V8,V9,Amount,Class

0,-1.359807,-0.072781,2.536346,1.378155,-0.338321,0.462388,0.239599,0.098698,0.363787,1
49.62,0

1,1.191857,0.266151,0.166480,0.448154,0.060018,-0.082361,-0.078803,0.085102,-0.255425,2
.69,0

2,-1.358354,-1.340163,1.773209,0.379780,-0.503198,1.800499,0.791461,0.247676,-1.514654,
378.66,1

Task 1 — Data Discovery using AWS Glue

Using **AWS Glue Console**

1. Create database

fraud_detection_db

2. Create a **Glue Crawler** that scans:

s3://fraud-detection-datalake/transactions/

3. The crawler should automatically create a table:

creditcard_transactions

Task 2 — Querying Data using AWS Athena

Use **Athena Console** to run the following queries.

Query 1 — Total Transactions

Find total number of transactions.

```
SELECT COUNT(*)  
FROM creditcard_transactions;
```

Query 2 — Fraud vs Normal Transactions

```
SELECT class, COUNT(*) as transaction_count
FROM creditcard_transactions
GROUP BY class;
```

Expected insight:

- Class = 0 → Normal
- Class = 1 → Fraud

Query 3 — Average Transaction Amount

```
SELECT
class,
AVG(amount) as avg_transaction
FROM creditcard_transactions
GROUP BY class;
```

Question to analyze:

Are fraud transactions higher value?

Query 4 — High Value Fraud Transactions

Find fraud transactions where amount > 200

```
SELECT *
FROM creditcard_transactions
WHERE class = 1
AND amount > 200;
```

Query 5 — Fraud Transactions per Time Window

Group by time ranges.

```
SELECT
CAST(time/3600 AS INTEGER) AS hour_window,
COUNT(*) as fraud_count
FROM creditcard_transactions
WHERE class = 1
GROUP BY CAST(time/3600 AS INTEGER)
ORDER BY fraud_count DESC;
```

Task 3 — Partition Optimization

Create a **partitioned Athena table** using

`class`

as partition column.

Store partitioned data in

`s3://fraud-detection-datalake/partitioned/`

Then compare query performance.

Task 4 — Python Program to Execute Athena Query

Write a **Python script using boto3** that:

1. Executes Athena query
2. Stores result in S3
3. Prints query output

Query:

```
SELECT class, COUNT(*)  
FROM fraud_detection_db.creditcard_transactions  
GROUP BY class;
```

Python should use:

- `boto3`
- `start_query_execution`
- `get_query_execution`
- `get_query_results`

Task 5 — AWS Glue ETL Job

Create a **Glue ETL Job using Python (PySpark)**.

Goal:

Create a new column:

risk_level

Rules:

amount < 50 → Low Risk

50 – 200 → Medium Risk

> 200 → High Risk

Output location:

s3://fraud-detection-datalake/processed-data/

Output format:

Parquet

Task 6 — Fraud Risk Analysis

Create Athena table for processed dataset.

Run query:

```
SELECT risk_level, COUNT(*) as transactions
FROM fraud_processed
GROUP BY risk_level;
```

Task 7 — Semi Structured Data Analysis

Assume another dataset arrives in JSON:

s3://fraud-detection-datalake/device-data/

Example JSON

```
{
  "transaction_id":12345,
```

```
"device_type":"mobile",  
"browser":"chrome",  
"ip_location":"USA",  
"fraud_flag":1  
}
```

Tasks:

1. Create Glue crawler
2. Create Athena table
3. Query

```
SELECT device_type, COUNT(*)  
FROM device_data  
WHERE fraud_flag = 1  
GROUP BY device_type;
```

Deliverables

Candidates should submit

1. Athena queries
2. Python script for Athena execution
3. Glue crawler screenshots
4. Glue ETL job code
5. Query results